

Cursor × ALT+TAB Hackathon — Kentsel CV Çözümü: Ürün, Veri Seti, Model ve Teknik Strateji Raporu

TL;DR

- Seç:** Google Street View tabanlı **yol hasarı** + (**zaman kalırsa**) **erişilebilirlik tespiti** — MVP'de tek/iki sınıf (pothole, opsiyonel crack). Hazır Hugging Face YOLO ağırlığı (`(peterhdd/pothole-detection-yolov8)`, Apache-2.0) + zorunlu **yüz/plaka blur gate** (`(deface)/CenterFace`). Hibrit demo: canlı inference + önceden hesaplanmış fixture fallback.
- Mimari:** `(masterfabric-go)` (Go 1.22+, Chi router, pgx/PostgreSQL, goose, JWT — Kafka kapalı) → ayrı Python CV servisi (Ultralytics YOLO + blur gate) → Render Postgres → Next.js harita/liste paneli (Vercel) + minimal read-only Expo ekranı. Cursor `(.cursor/rules)` ruleset + `(cursor-agent)` headless CLI ile README/KVKK rapor otomasyonu (MVP'yi riske atmayan bonus).
- KVKK kritik:** Ham (blur'suz) görüntü asla diske kalıcı yazılmaz, public repo/cloud'a gitmez; **blur, detection/eğitim/demodan ÖNCE** uygulanır; plaka/yüz modelleri SADECE anonimleştirme için (OCR/yüz tanıma/kimlik çıkarımı YASAK); ham veri demo sonrası silinir ve imha kaydı **en az 3 yıl** saklanır (KVKK Yönetmeliği md.7/3).

Executive Summary

6 saatlik, gerçek kentsel bağlamda kamu faydası üreten ve canlı demolanabilen en güçlü proje: **"YolGözü" — Google Street View görüntülerinden yol hasarı (çukur/çatlak) tespiti yapan, KVKK-uyumlu (yüz/plaka blur'lu) bir kentsel envanter ve harita paneli.** Bu fikir jüri puanlamasında dört kalemde birden öne çıkar: hazır YOLO modelleriyle **teknik çalışırlık (30p)**, street-level imgelerle eğitilmiş modeller sayesinde **CV doğruluğu (25p)**, belediye yol-bakım önceliklendirmesiyle **kamu faydası (20p)** ve güçlü blur-gate + imha kaydıyla **KVKK/etik (10p)**.

İkinci en güçlü alternatif **trafik/yönlendirme levha envanteri** (Mapillary MTSD / GTSIGN-220 / keremberke GTSDDB tabanlı), üçüncüsü **atık noktası/çöp tespiti** (TACO). Üçü de "anonimleştirme kanıtı" demosuyla KVKK puanını maksimize eder; ancak yol hasarı, hazır `(.pt)` ağırlık + en düşük teknik risk + en yüksek 6-saatte-bitirme olasılığı nedeniyle nettir.

Kritik tasarım kararı: Google ToS, Street View içeriğinin saklanması/cache'lenmesini yasaklar (yalnızca `(pano_id)` süresiz saklanabilir). Bu, KVKK'nın "ham görüntü cloud'a yüklenemez" kuralıyla aynı yöne işaret eder. Bu yüzden fixture olarak görüntü değil, **koordinat + heading/pitch/fov + pano_id** saklanmalı; görüntü demo anında çekilmeli.

Karşılaştırmalı Tablo — Fikir Matrisi (1-10)

#	Fikir	Tek. Yapılabilirlik	CV Doğruluk	Kamu Faydası	Demo Etkisi	KVKK Güvenliği	Veri/Model	6 Saatte Bitirme
1	Yol hasarı / çukur / çatlak	9	8	9	9	9	10	9
2	Trafik/yönlendirme levha	7	7	8	8	9	9	7
3	Atık noktası / çöp	7	6	9	8	9	8	7
9	Çok-sınıflı "kentsel kusur"	7	6	9	9	9	8	6
4	Kaldırım engeli / erişilebilirlik	5	6	10	9	9	5	5
5	Sokak lambası / kent mobilyası	6	6	7	6	9	6	6

Seçilen en iyi 3: (1) Yol hasarı, (2) Trafik levha, (3) Atık/çöp. **Neden:** Bu üçü hazır YOLO ağırlığına/COCO datasetine sahip, street-level imgelere transfer edilebilir, düşük KVKK riskli ve 6 saatte canlı demolanabilir. Erişilebilirlik (4) en yüksek "kamu faydası" hikâyesine sahip ama hazır object-detection modeli zayıf (Project Sidewalk ağırlıklı sınıflandırma/ResNet), bu yüzden 6 saat için riskli.

Bölüm 1 — Proje Fikri Adayları (8+)

1) Yol hasarı / çukur / asfalt çatlağı (ÖNERİLEN #1) — Kamu faydası: belediye yol-bakım önceliklendirmesi (yüksek). Street View çalışırılığı çok iyi (RDD2022 dashcam/street-level imgelerle eğitilmiş). HF kaynak bol. 6 saatte MVP yüksek. KVKK riski düşük (yol yüzeyi — yine de blur gate zorunlu). Demo etkisi yüksek. Teknik risk **düşük**.

2) Trafik/yönlendirme levha envanteri (ÖNERİLEN #2) — Eksik/hasarlı levha tespiti, envanter. HF: Mapillary MTSD (100K imge, 300+ sınıf), GTSIGN-220, keremberke GTSDb. MVP'de 300+ sınıfı "levha var/yok" tek sınıfa indir. Teknik risk **orta** (sınıf sayısı + transfer).

3) Atık noktası / çöp / döküntü (ÖNERİLEN #3) — Temizlik operasyonu, görsel olarak çarpıcı. HF/COCO: TACO (1.500 imge, 60 sınıf, TACO-10). Küçük objelerde zayıf (sigara izmariti <64px).

Teknik risk orta.

4) Kaldırım engeli / erişilebilirlik — curb ramp, missing curb ramp, obstacle, surface problem, no sidewalk. Project Sidewalk (GSV tabanlı, 700K+ etiket) ama hazır object-detection (.pt) sınırlı (ResNet-18/sınıflandırma). Kamu faydası **çok yüksek**, ama 6 saatte MVP **orta-düşük**.

5) Sokak lambası / direk / kent mobilyası — Cityscapes sınıfları (pole, traffic light, bench). Segmentation ağırlıklı; bbox demo orta. Risk orta.

6) Kaçak/uygunsuz tabela & görsel kirlilik — Etiketli hazır veri zayıf; genel obj-detection ile yaklaşık. Risk **orta-yüksek** (varsayım: özel fine-tune gerektirir, 6 saatte zor).

7) Yeşil alan / ağaç / gölgelik analizi — Cityscapes "vegetation" segmentasyonu + yeşil-alan oranı. Görsel çekici ama "doğruluk" ölçümü zor; demo etkisi orta.

8) Yaya geçidi / yol çizgisi solması — RDD2022 türevinde "Damaged Paint/Crosswalk" sınıfları var ama az örnek (orijinal Roboflow fork'ta bu 3 sınıf kapsam dışı bırakılmış). Risk orta.

9) (Daha iyi alternatif) Çok-sınıflı tek panel kentsel kusur — pothole + crack + çöp + hasarlı levha tek modelde. (Programmer-RD-AI/road-issues-detection-dataset) (9.660 imge) bunu destekler. Tek demoda çok kapsam; ama eğitim/entegrasyon 6 saatte risk artırır.

Bölüm 2 — En İyi 3 Fikrin Gerekçesi

- Yol hasarı (63 puan):** Tek satırla yüklenebilen Apache-2.0 ağırlık ((YOLO("https://huggingface.co/.../best.pt"))), street-level imge transferi, harita üstünde net bbox demosu. 30+25+20 puanlık kalemleri aynı anda hedefler.
 - Trafik levha (55):** Envanter hikâyesi güçlü, veri bol; ama fine-grained sınıf sayısı 6 saatte sadeleştirilmeli.
 - Atık/çöp (54):** Görsel olarak en "çarpıcı" demo, kamu faydası yüksek; küçük-obje doğruluğu zayıf olduğundan fixture seçimi kritik.
-

Bölüm 3 — Hugging Face Dataset & Model Listesi (gerçek linkler, lisanslar)

Yol hasarı

- (cvtechniques/road-damage-detection-yolov11) — YOLOv11s, lisans CC-BY-4.0, 4 sınıf (Alligator/Longitudinal/Transverse Crack + Pothole), RDD2022 (Japonya) tabanlı. ⚠️ **Türkiye/Avrupa transferi sınırlı** (model kartı açıkça "Geographically limited to Japan road infrastructure" diyor; pothole sınıfında veri az). <https://huggingface.co/cvtechniques/road-damage-detection-yolov11>

- `peterhdd/pothole-detection-yolov8` — YOLOv8s, **Apache-2.0**, 1 sınıf (pothole), 640×640, 100 epoch. **En kolay entegrasyon** + hazır Expo (react-native) örnek repo. **ÖNERİLEN MVP MODELİ**. <https://huggingface.co/peterhdd/pothole-detection-yolov8>
- `cazzz307/Pothole-Finetuned-YoloV8` — **Apache-2.0**, pothole (model kartı ">95% accuracy" iddia ediyor — **pazarlama ifadesi, doğrulanmamış**, kendi fixture'ınızda test edin). <https://huggingface.co/cazzz307/Pothole-Finetuned-YoloV8>
- `keremberke/yolov8n-pothole-segmentation` — pothole segmentation (instance mask). <https://huggingface.co/keremberke/yolov8n-pothole-segmentation>
- `OpenSistemas/YOLOv8-crack-seg` — YOLOv8 segmentation, Crack-seg (4.029 imge: 3.717 train / 200 val / 112 test). Yol+duvar çatlağı. <https://huggingface.co/OpenSistemas/YOLOv8-crack-seg>
- `Programmer-RD-AI/road-issues-detection-dataset` — 9.660 imge, çoklu sınıf (pothole, damaged road, broken sign, illegal parking, littering). Çok-sınıf panel için. <https://huggingface.co/datasets/Programmer-RD-AI/road-issues-detection-dataset>

Trafik levha

- `keremberke/german-traffic-sign-detection` — COCO formatı, 545 imge (GTSDDB). `load_dataset("keremberke/german-traffic-sign-detection")`. <https://huggingface.co/datasets/keremberke/german-traffic-sign-detection>
- `miriamcarnot/GTSIGN-220` — **CC-BY-SA-4.0**, 75.541 imge, Mapillary türevi (StVO-hızalı, fine-grained). ⚠ ShareAlike → türev eserler de CC-BY-SA. <https://huggingface.co/datasets/miriamcarnot/GTSIGN-220>
- `sparshgarg57/mapillary_traffic_signs` — Mapillary Vistas türevi. https://huggingface.co/datasets/sparshgarg57/mapillary_traffic_signs
- `nezahatkorkmaz/traffic-sign-detection` — YOLOv8, 30K+ imge. <https://huggingface.co/nezahatkorkmaz/traffic-sign-detection>
- Referans: Mapillary Traffic Sign Dataset (akademik kullanım serbest, 100K imge). <https://www.mapillary.com/dataset/trafficsign>

Atık / çöp

- **TACO** — COCO formatı, 1.500 imge, 60 sınıf (TACO-10), serbest lisans (atıf şartı). <http://tacodataset.org> · <https://github.com/pedropro/TACO>

Kent sahnesi / segmentation

- **Cityscapes** (HF: `huggan/cityscapes`, `tanganke/cityscapes`) — 5.000 ince + 20.000 kaba etiketli, 30 sınıf (pole, traffic light, bench, vegetation). <https://huggingface.co/datasets/huggan/cityscapes>

Anonimleştirme — SADECE blur amaçlı (OCR/kimlik YASAK)

- **deface** (ORB-HD) — CenterFace tabanlı, CLI: `deface input.jpg --replacewith blur|solid|mosaic`. Eşik (`--thresh`), default 0.2), maske ölçeği (`--mask-scale 1.3`). Canlı kamera modu da var. **ÖNERİLEN BLUR GATE**. <https://github.com/ORB-HD/deface>
- **khaclinh/pp4av** dataset — yüz + plaka anotasyonu (6 Avrupa şehri, gece dahil; YOLO formatı; class 0=face, 1=plate). Anonimleştirme benchmark'ı için. <https://huggingface.co/datasets/khaclinh/pp4av>
- Plaka tespiti (SADECE blur): morsetechlab/yolov11-license-plate-detection (⚠ model kartı upstream Roboflow datasetinde train/test sızıntısı olduğunu, metriklerin şişkin olduğunu açıkça belirtiyor — kullanırsanız kendi verinizde doğrulayın). <https://huggingface.co/morsetechlab/yolov11-license-plate-detection>
- **🚫 KESİNLİKLE KULLANMAYIN:** [MKgoud/License-Plate-Character-Detector](https://github.com/MKgoud/License-Plate-Character-Detector) ve benzeri OCR/karakter-okuma modelleri (plaka okuma = profileme/kimlik = KVKK ihlali ve hackathon kuralı ihlali).

Bölüm 4 — Google Street View Static API Kullanımı

Endpoint: [https://maps.googleapis.com/maps/api/streetview?](https://maps.googleapis.com/maps/api/streetview?size=640x640&location=LAT,LNG&heading=H&pitch=P&fov=F&key=KEY&signature=SIG)

[size=640x640&location=LAT,LNG&heading=H&pitch=P&fov=F&key=KEY&signature=SIG](https://maps.googleapis.com/maps/api/streetview?size=640x640&location=LAT,LNG&heading=H&pitch=P&fov=F&key=KEY&signature=SIG)

- **Maksimum görüntü boyutu 640×640 px** (Google resmi dokümanı: "Image sizes... are restricted to a maximum of 640x640 pixels"). Rate limit: 30.000 sorgu/dakika.
- Parametreler: `heading` 0-360 (0=Kuzey, 90=Doğu, 180=Güney), `pitch` -90..90, `fov` ≤120 (küçük fov = zoom), `radius` default 50m, `source=outdoor` (sadece dış mekan), `return_error_code=true` (imge yoksa gri imge yerine 404). (Google + 2)

Metadata endpoint (ÜCRETSİZ):

<https://maps.googleapis.com/maps/api/streetview/metadata?location=LAT,LNG&key=KEY>

→ JSON: `status` (OK / ZERO_RESULTS / NOT_FOUND), `pano_id`, `date`,

`location{lat,lng}`, `copyright`. **Önce metadata ile geçerli koordinatları filtreleyin** — boşa

ücretli istek atmayı önler. (Elvin Ouyang's Blog)

Fiyatlandırma: Pay-as-you-go. Google'ın resmi "Street View Static API Usage and Billing" dokümanına (güncelleme 2026-05-29) göre fiyat **hacme bağlı olarak panorama başına \$0.007 ile \$0.0056 arasında**. Ayrıca: tekrarlayan \$200 aylık kredi **1 Mart 2025'e kadar** geçerliydi, ardından "her API için ücretsiz kullanım eşiği" ile değiştirildi; Dinamik Street View Pro-tier SKU'su olup Pro SKU'lar aylık **5.000 ücretsiz** faturalanabilir olaya sahip. Demo için 5-20 panorama ≈ pratik olarak ücretsiz eşik içinde.

KRİTİK — Caching/saklama yasağı (Google ToS): "Content pre-fetching, indexing, storing, or caching is generally prohibited, except for place IDs and panorama IDs." Yani **görüntüyü saklayamazsınız; yalnızca (pano_id)'yi süresiz saklayabilirsiniz**. Bu, KVKK'nın "ham görüntü cloud/public repo'ya yüklenemez" kuralıyla aynı yöne işaret eder — ikisi birden tek bir tasarımla çözülür.

Tekrar üretilebilir demo fixture'ı (5-20 koordinat, İstanbul/Güngören): Görüntüyü değil; her nokta için `{lat, lng, heading, pitch, fov, pano_id}` JSON'unu saklayın. Demo anında görüntü çekilir → blur → detect. `(pano_id)` zamanla değişebileceğinden koordinatı koruyun ve çağrı anında `(pano_id)`'yi yenileyin (metadata `(ZERO_RESULTS)` dönerse koordinatı atlayın).

`(Google)`

Yüz/plaka blur — Street View'ın kendi blur'u yeterli mi? Hayır, defense-in-depth gerekli. Frome et al., Google Research, "Large-scale Privacy Protection in Google Street View" (ICCV 2009): "*we are able to sufficiently blur more than 89% of faces and 94–96% of license plates in evaluation sets sampled from Google Street View imagery.*" Yani yüzlerin ~%11'i, plakaların ~%4-6'sı kaçabilir → özellikle Güngören gibi yoğun yaya bölgelerinde **kendi blur gate'iniz (deface/CenterFace) zorunlu** ve demoda gösterilmeli.

Bölüm 5 — Teknik Mimari

Akış: Next.js panel (Vercel) → Go API (`(masterfabric-go)`, Render) → `(processing_jobs)` → Python CV servisi (Render: Ultralytics YOLO + deface blur gate) → Render Postgres → panelde harita/liste/anonimleştirme-kanıtı.

`(masterfabric-go)` gerçeği (repo incelendi):

- Tech stack: Go 1.22+, Chi HTTP router, PostgreSQL via pgx, Redis 7, Kafka opsiyonel (`(KAFKA_ENABLED=false)` bırakın — hackathon için gereksiz), goose migration, JWT + bcrypt, slog (JSON log), Prometheus/OpenTelemetry, go-playground/validator.
- Mimari: DDD/Clean/Hexagonal — bounded contexts: IAM, Tenant, API Management, Audit. Klasörler: `(cmd/server)`, `(internal/{shared, domain, application, infrastructure, gateway})`.
- Cursor hazır:** Repoda `(.cursor)` ve `(.cursor-plugin)` klasörleri **zaten mevcut** → ruleset bağlantısı doğal.
- Geliştirme: `(./dev.sh)` (Docker infra + goose migration + air hot-reload). Sunucu `(:8080)`, health: `(/health/live)`, `(/health/ready)`, `(/metrics)`.
- ⚠ **Lisans: AGPL v3.0** — ağ servisi olarak çalıştırırsanız değişikliklerinizi açık kaynak yapmanız gerekir. Hackathon repo'su zaten public olacağından sorun değil; README'de belirtin.

- API gateway pipeline'i (X-App-ID)/(X-Organization-ID) header'ı ile interceptor (PII masking dahil) destekliyor — bu, anonimleştirme adımını mimariye doğal bağlar.

Render Postgres şeması (önerilen):

```
image_sources(id, lat, lng, heading, pitch, fov, pano_id, source='streetview',
created_at)
processing_jobs(id, image_source_id FK, status, model_name, started_at,
finished_at)
anonymization_events(id, job_id FK, faces_blurred int, plates_blurred int,
method='centerface', verified bool, created_at)
detections(id, job_id FK, class, confidence, bbox_x, bbox_y, bbox_w, bbox_h,
created_at)
demo_runs(id, mode='live|fixture', job_count, created_at)
audit_events(id, actor, action, entity, detail jsonb, created_at) --
masterfabric-go audit zaten var
data_destruction_log(id, image_source_id FK, destroyed_at, method, operator) --
KVKK imha kaydı
```

API endpoint tasarımı:

- (POST /api/v1/jobs) (koordinat + heading/pitch/fov gönder → job oluştur)
- (GET /api/v1/jobs/{id}) (durum + sonuç)
- (GET /api/v1/detections?bbox=...) (harita için)
- (POST /api/v1/demo/run) (fixture toplu çalıştır)
- (GET /api/v1/audit) ve (GET /api/v1/destruction-log) (KVKK kanıt paneli)

Web sayfaları (Next.js): (1) Harita (MapLibre/Leaflet, tespit pinleri), (2) Tespit listesi/tablo (sınıf, confidence), (3) **Anonimleştirme kanıtı** (önce/sonra blur — ama "önce" sadece demo makinesinde, repoya gitmez), (4) KVKK/audit + imha kaydı paneli.

Expo minimum scope: Tek read-only ekran — koordinat seç → backend'e gönder → blur'lu sonuç + bbox göster.

Python worker mı, Go subprocess mı? Go'dan (os/exec) ile Python script çağırmak hızlı ama kırılgan. **Öneri:** ayrı **Python inference servisi** (Render'da ikinci web service, FastAPI; (peterhdd) örneğinde GPU'lu Docker + FastAPI hazır pattern var). Go API bu servise HTTP çağırır. Bu, masterfabric-go'yu saf kalır tutar ve hata yalıtımı sağlar.

⚠ **Render free tier uyarıları:** Free web service 15 dk inaktiflikten sonra spin-down olur, sonraki istekte 30-60 sn cold start → **demo öncesi servisleri ısıtın**. Free Postgres 1 GB ve **30 günde silinir** (hackathon için sorun değil ama bilin). GPU yok → YOLOv8n/s CPU inference'ı 640×640'ta saniyeler sürer, MVP için kabul edilebilir; fixture fallback bu yüzden şart. (Render + 2)

Bölüm 6 — Cursor SDK / CLI Bonus Stratejisi

Gerçek Cursor araçları (doğrulandı):

- Cursor CLI** — `curl https://cursor.com/install -fsS | bash`, komut `agent` (a.k.a. `cursor-agent`). Headless mod: `agent -p "..."`, `--output-format json|text`, `--force` (dosya değiştir). CI/script'lerde çalışır. `CURSOR_API_KEY` env ile.
- Cursor rules** — `.cursor/rules` dizini + proje kökünde `AGENTS.md`/`CLAUDE.md` otomatik okunur. (masterfabric-go'da `.cursor` zaten var.)
- Cursor SDK** — `npm install @cursor/sdk` (TypeScript, public beta), programatik agent + MCP + skills + hooks + subagents.

Düşük riskli öneriler (MVP'yi bloke etmeden 10 puanlık "AI/Cursor/agentik" kalemini hedefler):

Öneri	Uygulama zorluğu	README'de nasıl belgelenir	Ürün değeri	Bonus etkisi
<code>.cursor/rules</code> KVKK ruleset (yüz/plaka blur zorunlu, OCR yasak)	Düşük	Dosyayı README'ye link + ekran görüntüsü	Yüksek (gerçek guardrail)	Yüksek
<code>detections.json</code> → jüri özeti (<code>agent -p --output-format json</code>)	Düşük	Komut + örnek çıktı	Orta	Yüksek
Otomatik README/demo raporu (fixture sonuçlarından)	Düşük	Script + üretilen rapor	Orta	Orta
KVKK checklist quality-gate script (bash + cursor-agent)	Orta	CI adımı	Yüksek	Orta
Prompt log / agentik workflow belgesi	Düşük	<code>docs/agentik-workflow.md</code>	Orta	Orta

Strateji: Cursor işlerini **yan kanaldan** (build/rapor) çalıştırın; canlı demo path'ine sokmayın. 5:15-6:00 slotunda yapılır.

Bölüm 7 — KVKK ve Etik Güvenlik Planı

Hukuki temel: KVKK md.6'ya göre biyometrik veriler özel nitelikli kişisel veridir. Ancak bir

fotoğraf **tek başına** biyometrik veri değildir; yalnızca "spesifik teknik işleme" (yüz tanıma/eşleştirme, kimlik çıkarımı) ile biyometrik nitelik kazanır (Kurul 25/03/2019 ve 2019/81 sayılı kararlar; GDPR md.4/14 ile uyumlu). **Bizim modelimiz SADECE blur amaçlı çalışır, hiçbir kimlik/eşleştirme feature'ı üretmez** — bu, KVKK'nın biyometrik-veri rejimine girmemenin anahtarıdır.

Pipeline kuralı (en kritik tasarım): Street View görüntüsü gelir → **ÖNCE blur gate (yüz + plaka tespiti → Gaussian/solid mask)** → ANCAK ondan SONRA yol-hasarı detection modeli görüntüyü görür ve sonuçları depolar. Ham (blur'suz) görüntü **diske kalıcı yazılmaz**, public repo/cloud'a **gitmez**, sadece RAM/geçici tampon.

Anonimleştirme standardı (geri döndürülemezlik): KVKK "Kişisel Verilerin Silinmesi, Yok Edilmesi veya Anonim Hale Getirilmesi" Yönetmeliği md.10'a göre anonimleştirme, "kişisel verilerin, veri sorumlusu, alıcı veya alıcı grupları tarafından **geri döndürme** ve verilerin başka verilerle eşleştirilmesi gibi... uygun tekniklerin kullanılması yoluyla **dahi** kimliği belirli veya belirlenebilir bir gerçek kişiyle ilişkilendirilemez hale getirilmesi"dir. KVKK Rehberi bu tersine-çevrilebilirliği "anonimliğin bozulması" olarak adlandırır. ⚠ **Teknik not:** Gaussian blur teorik olarak deblurring saldırılarıyla kısmen geri çevrilebilir (HF topluluk tartışmalarında belirtiliyor). Bu yüzden MVP'de **solid mask veya güçlü Gaussian + downscale** kullanın; demoda "geri döndürülemez maskeleyme" ifadesini bilinçli seçin. KVKK Rehberi'nde maskeleyme açıkça bir anonimleştirme tekniği olarak sayılır ("Maskeleyme: kişisel verilerin belli alanlarının... silinmesi, üstlerinin çizilmesi, boyanması..."). (Kvkk) (Bahukuk)

İmha belgeleme (zorunlu): Yönetmelik md.7/(3) — "Kişisel verilerin silinmesi, yok edilmesi ve anonim hale getirilmesiyle ilgili yapılan bütün işlemler kayıt altına alınır ve söz konusu kayıtlar, diğer hukuki yükümlülükler hariç olmak üzere **en az üç yıl süreyle saklanır.**" Pratik karşılığı: (data_destruction_log) tablosu + README'de "Veri Saklama ve İmha" bölümü + hackathon sonu (make destroy-raw) script'i (ham tamponları siler, log yazar). md.7/(4): kullanılan imha yöntemleri politika/prosedürde açıklanmalı. (Kvkk) (Kvkk)

Demoda gösterilecek: blur önce/sonra (yalnızca demo makinesinde), audit log, "ham veri silindi" imha kaydı, "OCR/yüz tanıma yok" beyanı. **Demoda GÖSTERİLMEYECEK:** blur'suz yüz/plaka, herhangi bir plaka metni, kişi/araç takibi, profillemeye.

Riskli edge case'ler: (a) Yoğun kalabalıkta missed face → eşiği düşür ((--thresh 0.15)), maske ölçeğini büyüt ((--mask-scale 1.5)); (b) Street View'da insanlar yansıma/vitrin içinde → genel (person) sınıfını da blur'la; (c) ev/iş yeri tabelasındaki isim/telefon → MVP'de OCR yok zaten, ama tam-frame demo görüntüsünü repoya koymayın.

1. **Street View Türkiye kapsama boşlukları** → metadata pre-check (`(status:OK)` filtresi); fixture koordinatlarını önceden doğrula.
2. **Japonya-eğitilmiş modelin Türkiye transferi zayıf** (RDD2022 model kartı bunu açıkça söylüyor) → confidence eşikini düşür, demo fixture'ını model iyi çalıştığı sahnelerden seç; mümkünse Apache-2.0 pothole modelini tercih et.
3. **Render free cold start 30-60 sn** → demo öncesi ısıt; canlı inference patlarsa fixture fallback.
4. **Render free Postgres 30 günde silinir** → hackathon için sorun değil, README'de not.
5. **Google caching ToS** → görüntü saklama, sadece pano_id/koordinat sakla.
6. **Lisans:** masterfabric-go AGPL-3.0 (repo public olacak, uyumlu); GTSIGN-220 CC-BY-SA (türev de SA); TACO atıf şartı; pothole modelleri Apache-2.0 (en rahat).
7. **Şişkin metrik uyarısı:** Bazı model kartları (" $>95\%$ accuracy", license-plate train/test sızıntısı) pazarlama/şişkin metrik içeriyor → kendi fixture'ınızda doğrulayın, jüriye "kendi test sonucumuz" deyin.

6 Saatlik Uygulama Planı (iyileştirilmiş + kritik path)

Saat	İş	Not
0:00-0:30	Scaffold: masterfabric-go fork + <code>(./dev.sh)</code> ; Render Postgres + Vercel proje + GSV API key + HF token env	Kritik path başlangıcı; Kafka kapalı
0:30-1:30	CV anonymization blur gate (deface/CenterFace) çalışır + test imgesi	★ KRİTİK PATH — bu olmadan demo yok
1:30-2:30	Object detection: <code>(peterhdd/pothole)</code> <code>(.pt)</code> indir, inference script, blur sonrası bbox JSON	
2:30-3:30	Go API: jobs/detections endpoint + goose migration + audit	masterfabric-go pattern'ini izle
3:30-4:30	Next.js UI: harita + liste + anonimleştirme kanıtı sayfası	
4:30-5:15	Deploy (Render Go + Python, Vercel Next) + fixture fallback hazırla	Fallback bu noktada hazır olmalı
5:15-6:00	README + KVKK/imha bölümü + Cursor ruleset/CLI rapor + sunum provası	Bonus + 5p README

Kritik path: (blur gate → inference → JSON → UI render). Fixture fallback (önceden hesaplanmış detection JSON + blur'lu imge) **4:30'da** hazır olmalı ki canlı inference patlarsa demo kurtulsun.

Demo Senaryosu (3-5 dk)

1. **Problem (30sn):** "Belediyeler yol/kaldırım kusurlarını manuel tarıyor — yavaş ve pahalı."
 2. **Veri girişi (30sn):** Güngören'den bir koordinat seç → Street View görüntüsü canlı çekilir.
 3. **Anonimleştirme kanıtı (45sn):** Önce/sonra — yüz/plaka blur'lu. "Ham görüntü asla saklanmaz, hiç diske yazılmaz."
 4. **Obje tespiti (45sn):** Pothole/crack bbox + confidence skoru.
 5. **Harita/liste (30sn):** Tespitler haritada pinlenir, liste tablosu.
 6. **Kamu faydası (30sn):** "Otomatik bakım önceliklendirme listesi."
 7. **KVKK (30sn):** audit log + imha kaydı + "OCR/yüz tanıma kesinlikle yok."
 8. **AI/Cursor (20sn):** (.cursor/rules) KVKK guardrail + cursor-agent otomatik rapor.
 9. **Tekrarlanabilirlik (20sn):** Aynı koordinat fixture'ı → aynı sonuç (canlı patlar fallback).
-

Nihai Tavsiye

- **Proje fikri:** Google Street View tabanlı **yol hasarı (pothole)** + **opsiyonel çatlak tespiti**, harita panelli kentsel envanter.
- **Model:** (peterhdd/pothole-detection-yolov8) (Apache-2.0, tek sınıf, tek satır yükleme) + blur gate (deface) (CenterFace).
- **MVP sınıfları:** (pothole) (kesin); zaman kalırsa (crack) (longitudinal/transverse).
- **Kapsam dışı (net):** plaka OCR/karakter okuma, yüz tanıma/eşleştirme, kişi/araç takibi, profillemeye, 300+ fine-grained levha sınıfı, segmentation tabanlı yeşil-alan analizi.
- **Mimari:** masterfabric-go (Chi+pgx+goose+JWT, Kafka kapalı) + ayrı Python FastAPI inference servisi + Render Postgres + Next.js/Vercel + minimal read-only Expo.
- **Dikkat:** Street View kapsama (metadata pre-check), model transfer zayıflığı (eşik + fixture), Render cold start (ısıtma + fallback), caching ToS (pano_id sakla, görüntü değil), AGPL/CC-BY-SA lisans notları.

İlk yapılacak 10 somut iş:

1. masterfabric-go fork + (.dev.sh) ile local çalıştır (Postgres+Redis hazır, Kafka kapalı).
2. Render Postgres + Vercel projesi + Google Street View API key + Hugging Face token env ayarla.
3. (.cursor/rules)'a KVKK kurallarını yaz ("yüz/plaka blur zorunlu, OCR/identity yasak").

4. `deface` blur gate Python script'i (yüz + plaka → solid/Gaussian mask).
 5. `peterhdd/pothole-detection-yolov8` `.pt` indir + inference script.
 6. Blur → detect pipeline'ını birleştir, **blur sonrası** detection JSON çıktısı üret.
 7. Go migration (image_sources, processing_jobs, detections, anonymization_events, data_destruction_log, demo_runs).
 8. Go endpoints + Python inference servisi HTTP bağlantısı.
 9. Next.js harita (MapLibre) + tespit listesi + anonimleştirme kanıtı sayfası.
 10. 5-20 koordinatlık İstanbul/Güngören fixture (lat,lng+heading+pitch+fov+pano_id) + fixture fallback + README KVKK/imha bölümü + cursor-agent jüri raporu.
-

Varsayım notları: Hackathon tarihi/kuralları kullanıcı brief'inden alınmıştır. Model doğruluk iddiaları (">95%") doğrulanmamış pazarlama ifadeleridir; kendi fixture'ınızda ölçün. Street View fiyat/kota değerleri Google'ın 2026-05-29 güncel dokümanına dayanır, demo tarihinde teyit edin. Render free tier limitleri Mayıs 2026 itibarıyladır.